interpretnn: An R Package for Statistically-Based Neural Network Interpretation

by Andrew McInerney and Kevin Burke

Abstract Feedforward neural networks are widely available in R through packages such as **nnet**, **neuralnet**, **keras**, and **torch**. However, the output from these packages is typically prediction-oriented and does not resemble the regression-style summaries familiar to many statisticians. The **interpretnn** package provides a post-processing framework for fitted feedforward neural-network objects in R. It converts supported neural-network fits into a common `"interpretnn"` object and provides S3 methods for prediction, summaries, Wald-type tests, partial covariate-effect plots with uncertainty estimates, and neural-network architecture diagrams annotated with inferential information. The package is designed to make shallow feedforward neural networks more accessible as statistical modelling tools, while retaining compatibility with existing neural-network fitting software. This paper introduces the statistical quantities computed by **interpretnn**, describes the package interface and implementation, and demonstrates a typical workflow using an applied regression example.

1 Introduction

Feedforward neural networks (FNNs) are flexible non-linear models that are widely used for prediction and function approximation. In R, several packages are available for fitting FNNs, including **nnet** (Venables and Ripley, 2002), **neuralnet** (Fritsch et al., 2019), **keras** (Allaire and Chollet, 2023), and **torch** (Falbel and Luraschi, 2023). These packages provide flexible tools for model fitting and prediction, but their outputs are typically designed from a machine-learning perspective. For example, fitted neural-network objects usually provide fitted values, predictions, training losses, and access to model weights, but they do not generally provide regression-style summaries such as standard errors, Wald-type tests, analysis-of-variance-style tables, or covariate-effect summaries.

This creates a gap for statisticians who wish to use shallow FNNs as statistical modelling tools rather than purely predictive algorithms. In classical regression modelling, users often rely on familiar generic functions such as `summary()`, `aov()`, `predict()`, and `plot()` to inspect a fitted model, assess the contribution of covariates, and communicate estimated effects. Comparable outputs are not routinely available for FNNs, even though shallow neural networks can be written as non-linear regression models with likelihood-based objectives. Consequently, users who want to interpret a fitted FNN must often extract weights manually, write custom code for model summaries, or use model-agnostic interpretability tools that do not exploit the specific neural-network parameterisation.

The statistical methodology underlying the approach implemented in **interpretnn** is developed in McInerney and Burke (2023). That work treats suitably parsimonious shallow FNNs as statistical models, investigates the performance of Wald-type tests for individual weights and covariate-level groups of weights, and introduces partial covariate-effect plots as neural-network analogues of regression coefficients. The present article focuses on the corresponding R implementation. In particular, we describe how these statistically motivated summaries can be made available through familiar R syntax and standard S3 methods.

The **interpretnn** package addresses the gap between neural-network fitting software and regression-style statistical output by providing a statistical post-processing layer for fitted FNNs in R. The package does not introduce a new neural-network optimiser. Instead, it works with neural networks fitted using existing R packages and converts them into a common `"interpretnn"` object. This object can then be used with familiar S3 methods. The aim is to make fitted FNNs behave more like standard statistical model objects in R, while preserving compatibility with existing neural-network fitting software.

The main functionality of **interpretnn** includes regression-style summaries for fitted FNNs; Wald-type tests for individual weights, input nodes, and categorical covariates; partial covariate-effect plots with uncertainty estimates; prediction methods for new observations; and neural-network architecture diagrams annotated with the results of the statistical summaries. The package also includes a convenience function, `nn\_fit()`, for fitting simple FNNs using a familiar formula interface before applying the interpretation tools.

The inferential quantities produced by **interpretnn** should be interpreted as approximate, model-based summaries of a fitted neural network. As with other non-linear models, the validity of Wald-

type summaries depends on regularity conditions and on the stability of the fitted model. For neural networks in particular, issues such as hidden-node symmetries, redundant hidden nodes, local optima, and near-singular Hessian matrices can affect parameter-based inference. Accordingly, **interpretnn** is intended for relatively shallow, parsimonious FNNs, and the resulting summaries should be used as statistically motivated diagnostic and interpretive tools rather than as definitive inferential guarantees for arbitrary neural-network architectures.

The remainder of the paper is organised as follows. Section 2.2 briefly introduces the FNN parameterisation and the statistical quantities used by **interpretnn**. Section 2.3 describes the main package workflow and the structure of `"interpretnn"` objects. Section 2.4 demonstrates the package using an applied regression example. Section 2.5 describes the supported model objects and S3 methods. Section 2.7 discusses practical limitations and appropriate interpretation of the resulting summaries. Section 2.8 concludes.

2 Statistical background

This section briefly summarises the statistical quantities computed by **interpretnn**. A fuller development of the methodology, including the derivation of the Wald-type summaries, simulation evidence, and practical recommendations, is given in McInerney and Burke (2023). Here, the aim is to introduce only the notation and definitions needed to understand the package output.

The **interpretnn** package is designed for shallow feedforward neural networks that can be viewed as non-linear regression models. Let y_i denote the response for observation $i = 1, \ldots, n$, and let $x_i = (x_{i0}, x_{i1}, \ldots, x_{ip})^T$ denote the corresponding vector of covariates, where $x_{i0} \equiv 1$ is an intercept term. For a single-hidden-layer FNN with q hidden nodes, the conditional mean can be written as

$$\mathbb{E}(y_i \mid x_i) = \mathrm{NN}(x_i, \theta),$$

where

$$\mathrm{NN}(x_i, \theta) = \phi_o \left[\gamma_0 + \sum_{k=1}^{q} \gamma_k \phi_h \left(\sum_{j=0}^{p} \omega_{jk} x_{ij} \right) \right].$$

Here, ω_{jk} is the weight connecting input j to hidden node k, γ_k is the weight connecting hidden node k to the output node, γ_0 is the output intercept, and θ denotes the full vector of model parameters. The functions $\phi_h(\cdot)$ and $\phi_o(\cdot)$ are the hidden-layer and output-layer activation functions, respectively. For continuous responses, $\phi_o(\cdot)$ is typically the identity function; for binary responses, it is typically the logistic function.

The package works with fitted FNNs and computes model-based summaries from the fitted parameter vector, the model matrix, the response, and the assumed response distribution. For continuous responses, **interpretnn** uses a Gaussian likelihood; for binary responses, it uses a Bernoulli likelihood. When a ridge penalty is used during fitting, the penalised log-likelihood can be written as

$$\ell_\lambda(\theta) = \sum_{i=1}^{n} \log f(y_i \mid x_i, \theta) - \lambda \|\tilde{\theta}\|_2^2,$$

where $f(y_i \mid x_i, \theta)$ is the assumed density or probability mass function, λ is the ridge penalty, and $\tilde{\theta}$ denotes the vector of non-intercept neural-network weights. This corresponds to the weight-decay penalty commonly used when fitting neural networks. The fitted parameter vector and an estimated covariance matrix are then used to construct the regression-style summaries, Wald-type tests, and partial covariate-effect plots described below.

Wald-type summaries

One aim of **interpretnn** is to provide regression-style summaries for fitted FNNs. Given an estimate $\hat{\theta}$ and an estimated covariance matrix $\widehat{\mathrm{Cov}}(\hat{\theta})$, the package computes Wald-type summaries for both individual weights and groups of weights.

For an individual parameter θ_j, the null hypothesis

$$H_0 : \theta_j = 0$$

is summarised using the statistic

$$W_j = \frac{\hat{\theta}_j^2}{\widehat{\mathrm{Cov}}(\hat{\theta})_{jj}},$$

which is compared to a χ^2_1 reference distribution. This produces a parameter-level summary similar in form to the output from classical regression models. However, individual neural-network weights are often difficult to interpret in isolation, because the effect of an input variable is distributed across multiple connections.

For this reason, **interpretnn** also provides covariate-level Wald-type summaries. For a single-hidden-layer FNN, the contribution of covariate x_j to the hidden layer is represented by the vector

$$\omega_j = (\omega_{j1}, \ldots, \omega_{jq})^T,$$

which contains all weights connecting covariate j to the hidden nodes. The package therefore computes a multiple-parameter Wald-type statistic for

$$H_0 : \omega_j = 0_q.$$

If S_j is the matrix selecting the elements of θ corresponding to ω_j, then the relevant covariance matrix is

$$\widehat{\Sigma}_{\omega_j} = S_j \widehat{\mathrm{Cov}}(\hat{\theta}) S_j^T.$$

The corresponding statistic is

$$W_j = \hat{\omega}_j^T \widehat{\Sigma}_{\omega_j}^{-1} \hat{\omega}_j.$$

This provides an input-level summary that is closer in spirit to testing whether a covariate contributes to the fitted model.

For categorical covariates represented by several dummy variables, **interpretnn** extends this idea by testing all associated input nodes jointly. For example, if a categorical covariate with $l + 1$ levels is represented using l dummy variables, and the network has q hidden nodes, the corresponding grouped test involves lq input-to-hidden weights. This gives an analysis-of-variance-style summary for categorical predictors, allowing the user to assess the contribution of the original covariate rather than each dummy variable separately.

The Wald-type quantities reported by **interpretnn** are approximate, model-based summaries of a fitted FNN. Their interpretation depends on the stability of the fitted model and on the regularity of the local parameterisation. In particular, neural networks can exhibit hidden-node symmetries, redundant nodes, local optima, and near-singular Hessian matrices. These issues are discussed in detail in McInerney and Burke (2023). In the present article, we focus on how these summaries are computed and exposed through the package interface.

Partial covariate effects

In addition to assessing whether a covariate contributes to the fitted model, it is useful to describe the nature of that contribution. For this purpose, **interpretnn** provides partial covariate-effect plots.

Partial dependence plots describe how the average fitted response changes as one covariate is varied while the remaining covariates are held at their observed values. For covariate j, the partial dependence function is estimated as

$$\overline{\mathrm{NN}}(x^{(j)}) = \frac{1}{n} \sum_{i=1}^{n} \mathrm{NN}(x_i^{(j)}, \hat{\theta}),$$

where $x_i^{(j)}$ denotes the covariate vector for observation i with the jth covariate set to the value $x^{(j)}$.

The partial covariate effect (PCE) is a finite-difference version of this quantity. For a d-unit increase in covariate j, the PCE is defined as

$$\hat{\beta}(x^{(j)}, d) = \overline{\mathrm{NN}}(x^{(j)} + d) - \overline{\mathrm{NN}}(x^{(j)}).$$

This quantity is designed to provide an interpretation closer to a regression coefficient: it estimates the average change in the fitted response associated with a d-unit increase in a covariate. The value of d can be chosen by the user; common choices are $d = 1$ or d equal to one empirical standard deviation of the covariate.

For binary covariates, the PCE corresponds to the average change in the fitted response when the covariate changes from 0 to 1. For continuous covariates, plotting $\hat{\beta}(x^{(j)}, d)$ over a sequence of values shows how the covariate effect varies across the covariate range. When the PCE is approximately constant, the fitted relationship is approximately linear in that covariate; when it varies substantially, the fitted model suggests a non-linear effect.

Uncertainty intervals for the PCE can be computed using the approximate sampling distribution

of $\hat{\theta}$. For the delta method, the gradient of the PCE with respect to $\hat{\theta}$ is used:

$$\nabla_{\hat{\theta}}\hat{\beta}(x^{(j)}, d) = \frac{1}{n}\sum_{i=1}^{n}\left[\nabla_{\hat{\theta}}\mathrm{NN}(x_i^{(j)} + d, \hat{\theta}) - \nabla_{\hat{\theta}}\mathrm{NN}(x_i^{(j)}, \hat{\theta})\right].$$

The package also provides a single point-estimate covariate effect by averaging the PCE over the observed values of the covariate:

$$\hat{\tau}_j = \frac{1}{n}\sum_{i=1}^{n}\hat{\beta}(x_{ij}, d).$$

This value is reported in the package summary output and is accompanied by a standard error. Together, the Wald-type summaries and PCE plots allow **interpretnn** to provide both of the main pieces of information usually sought from regression-style output: whether a covariate appears to contribute to the fitted model, and how the fitted response changes with that covariate.

3 Package workflow

The main function in **interpretnn** is `interpretnn()`. It takes an existing fitted neural-network object and converts it into an object of class `"interpretnn"`:

```
interpretnn(object, ...)
```

The resulting object stores the fitted weights, model matrix, response, likelihood information, covariance estimates, Wald-type summaries, covariate-effect estimates, and other quantities required by the package methods. The purpose of this object is to provide a common interface for obtaining statistical summaries from fitted FNNs.

A typical workflow is:

```
# 1. Fit a neural network
nn <- nn_fit(
  formula = charges ~ .,
  data = insurance,
  q = 2,
  n_init = 10,
  lambda = 0.01,
  pkg = "nnet"
)

# 2. Convert to an interpretnn object
intnn <- interpretnn(nn)

# 3. Inspect the fitted model
print(intnn)
summary(intnn)
aov(intnn)

# 4. Visualise the fitted model
plot(intnn, which = c("age", "smoker"), conf_int = TRUE)
plotnn(intnn)

# 5. Predict for new observations
predict(intnn, newdata = insurance[1:5, ])
```

Although **interpretnn** provides the convenience function `nn_fit()`, the package is not primarily a neural-network fitting package. Rather, it is designed as a post-processing and interpretation layer for fitted FNNs. Where possible, users can fit their neural network using an existing R package and then pass the fitted object to `interpretnn()`.

Fitting a neural network

For convenience, **interpretnn** includes the function `nn_fit()`, which fits a shallow FNN using a formula interface. This allows users to fit a model using syntax that is close to standard modelling functions in R:

Table 1: Main components of an object of class 'interpretnn'.

Component	Description
'weights'	Vector of fitted neural-network parameters.
'loss_val'	Value of the relevant loss function.
'n_inputs'	Number of input nodes.
'n_nodes'	Number of hidden nodes in each hidden layer.
'n_layers'	Number of hidden layers.
'n_param'	Number of model parameters.
'n'	Number of observations in the training data.
'loglike'	Value of the model log-likelihood.
'BIC'	Bayesian information criterion for the fitted model.
'eff'	Point-estimate partial covariate effects and standard errors.
'wald'	Multiple-parameter Wald-type summaries for input nodes and covariates.
'wald_sp'	Single-parameter Wald-type summaries for individual weights.
'X'	Model matrix used by the fitted neural network.
'y'	Response vector.
'response'	Response type, such as continuous or binary.
'lambda'	Ridge penalty used in fitting.

```
library(interpretnn)

set.seed(1237019)

nn <- nn_fit(
  formula = charges ~ .,
  data = insurance,
  q = 2,
  n_init = 10,
  lambda = 0.01,
  pkg = "nnet"
)
```

In this example, the response is `charges`, all remaining variables in `insurance` are used as covariates, and the fitted FNN has two hidden nodes. The argument `n_init` controls the number of random initialisations used during fitting, while `lambda` controls the strength of the ridge penalty. The argument `pkg` specifies the package used to fit the neural network.

The `nn_fit()` function is intended to make **interpretnn** easy to use for standard examples and simple workflows. However, users who require more control over optimisation, architecture, or training can fit the neural network directly using a supported package and then apply `interpretnn()` to the resulting fitted object.

Creating an "interpretnn" object

Once a neural network has been fitted, the `interpretnn()` function converts it into an object of class `"interpretnn"`:

```
intnn <- interpretnn(object = nn)
```

If the model was fitted using `nn_fit()`, the fitted object already contains the model information needed by `interpretnn()`. If the neural network was fitted directly using another package, additional arguments may be required. For example, some fitted objects do not store the original model formula or training data, so these may need to be supplied separately.

The object returned by `interpretnn()` contains the main quantities used by the package methods. The most important components are described in Table **??**.

The user will not usually need to access these components directly. Instead, they are used internally by the S3 methods described below.

Printing and summarising the fitted network

The `print()` method provides a compact description of the fitted network, including the model call and the neural-network architecture:

```
print(intnn)
```

The `summary()` method provides the main regression-style summary of the fitted FNN:

```
summary(intnn)
```

The summary output reports, for each covariate, the point-estimate partial covariate effect, its standard error, the multiple-parameter Wald-type statistic, and the corresponding p-value. This gives a compact overview of both the estimated covariate effect and the evidence that the covariate contributes to the fitted FNN.

By default, the summary focuses on input-level and covariate-level summaries. Single-parameter Wald-type summaries for individual neural-network weights can also be displayed:

```
summary(intnn, wald_single_par = TRUE)
```

These individual-weight summaries are mainly diagnostic, since individual neural-network weights are generally less interpretable than input-level or covariate-level summaries.

Analysis-of-variance-style summaries

The `aov()` method provides covariate-level Wald-type summaries:

```
aov(intnn)
```

For continuous or binary covariates, this is equivalent to the input-level multiple-parameter Wald-type summary reported by `summary()`. For categorical covariates represented by multiple dummy variables, `aov()` tests the associated input nodes jointly. This provides a summary closer in spirit to an analysis-of-variance table for standard regression models, where the contribution of the original covariate is assessed rather than the contribution of each dummy variable separately.

Visualising the fitted network

The `plotnn()` function visualises the fitted neural-network architecture and annotates it using the Wald-type summaries:

```
plotnn(intnn)
```

By default, statistically significant weights and input nodes are highlighted, while non-significant components are displayed in a lighter colour. The significance threshold can be controlled using the `alpha` argument, and colours can be changed using `col.sig` and `col.insig`. Intercept terms are omitted by default but can be displayed using `intercept = TRUE`.

Covariate-effect plots

The `plot()` method produces partial covariate-effect plots:

```
plot(intnn, which = c("age", "smoker"), conf_int = TRUE)
```

The argument `which` selects the covariates to be plotted. If `conf_int = TRUE`, uncertainty intervals are added to the plot. The uncertainty method can be specified using the `method` argument:

```
plot(
  intnn,
  which = "age",
  conf_int = TRUE,
  method = "delta_method"
)
```

Alternatively, simulation from the approximate sampling distribution of the fitted parameters can be used:

```
plot(
  intnn,
  which = "age",
  conf_int = TRUE,
  method = "mlesim"
)
```

For continuous covariates, the PCE plot shows how the estimated covariate effect varies across the covariate range. For binary covariates, it displays the estimated change in fitted response when the covariate changes from 0 to 1. This allows the user to inspect not only whether a covariate appears to contribute to the fitted network, but also how the fitted response changes with that covariate.

Prediction

The `predict()` method returns fitted values for the training data or predictions for new observations:

```
predict(intnn)
```

```
predict(intnn, newdata = insurance[1:5, ])
```

This allows an `"interpretnn"` object to be used in a similar way to other fitted model objects in R.

Summary of the workflow

Overall, the package workflow is designed to mirror standard statistical modelling practice in R. A neural network is first fitted, either using `nn_fit()` or an external neural-network package. The fitted object is then converted using `interpretnn()`, after which standard methods such as `print()`, `summary()`, `aov()`, `plot()`, and `predict()` can be used. This workflow allows users to inspect fitted FNNs using tools that are familiar from regression modelling, while retaining the flexibility of neural-network fitting packages.

4 Applied example

This section demonstrates the main functionality of **interpretnn** using the insurance data considered in **?**. The aim is not to repeat the full applied analysis from that paper, but to show how the package can be used to obtain regression-style summaries, covariate-level tests, PCE plots, network visualisations, and predictions from a fitted shallow FNN.

Data

The data contain information on $n = 1338$ insurance beneficiaries. The response variable, `charges`, is the total medical expenses charged to an insurance plan. The explanatory variables are the beneficiary's age, sex, body mass index, number of children covered by the plan, smoking status, and region of residence.

```
data("insurance", package = "interpretnn")
```

```
str(insurance)
```

For this example, we model `charges` using all available covariates:

```
formula <- charges ~ age + sex + bmi + children + smoker + region
```

The variables include both continuous covariates, such as `age` and `bmi`, and categorical covariates, such as `sex`, `smoker`, and `region`. This makes the data useful for illustrating both input-level summaries and covariate-level summaries for factors represented by multiple dummy variables.

Fitting the neural network

We first fit a single-hidden-layer FNN using the convenience function `nn_fit()`:

```
library(interpretnn)

set.seed(1237019)

nn <- nn_fit(
  formula = charges ~ age + sex + bmi + children + smoker + region,
  data = insurance,
  q = 2,
  n_init = 10,
  lambda = 0.01,
  pkg = "nnet"
)
```

The fitted model has two hidden nodes and uses a ridge penalty with $\lambda = 0.01$. The argument `n_init = 10` specifies that ten random initialisations are used during fitting, with the best fitted model retained. This is useful because neural-network optimisation can depend on the initial parameter values.

The object `nn` contains the fitted network and the additional model information required by **interpretnn**. It can then be converted into an `"interpretnn"` object:

```
intnn <- interpretnn(nn)
```

Printing the fitted network

The `print()` method gives a compact overview of the fitted FNN:

```
print(intnn)
```

The printed output reports the model call, response type, number of observations, number of input nodes, number of hidden nodes, and the value of the ridge penalty. This provides a quick check that the fitted network has the intended architecture before examining the statistical summaries.

Regression-style summary

The `summary()` method gives the main regression-style summary produced by **interpretnn**:

```
summary(intnn)
```

The output contains one row for each original covariate. For each covariate, **interpretnn** reports a point-estimate partial covariate effect, its standard error, a Wald-type statistic, and a corresponding p-value. Thus, the summary combines two pieces of information that are commonly sought in regression modelling: an estimate of the size of the covariate effect and a test of whether the covariate appears to contribute to the fitted model.

For example, in the insurance data, variables such as `age`, `bmi`, `children`, and `smoker` are expected to contribute strongly to the fitted response. The summary output allows these effects to be inspected in a form that is closer to a classical regression summary than to the raw weight output usually obtained from neural-network software.

By default, `summary()` focuses on covariate-level summaries. Single-parameter summaries for individual neural-network weights can also be displayed:

```
summary(intnn, wald_single_par = TRUE)
```

These individual-weight summaries are mainly diagnostic. For interpretation, the covariate-level summaries and the PCE plots are usually more informative than individual input-to-hidden-layer weights.

Analysis-of-variance-style summary

The `aov()` method provides covariate-level Wald-type summaries:

```
aov(intnn)
```

For continuous and binary covariates, this corresponds to testing the group of weights connecting the covariate to the hidden layer. For categorical covariates represented by multiple dummy variables, such as `region`, the method tests all associated input nodes jointly. This gives a summary that is analogous in spirit to an analysis-of-variance table, where the contribution of the original covariate is assessed rather than the contribution of each dummy variable separately.

This is particularly useful when interpreting factor variables. For example, rather than separately inspecting the dummy variables corresponding to different regions, the `aov()` method gives a single covariate-level summary for `region`.

Network visualisation

The fitted architecture can be visualised using `plotnn()`:

```
plotnn(intnn)
```

The plot displays the input nodes, hidden nodes, output node, and fitted connections. By default, statistically significant input nodes and weights are highlighted, while non-significant components are shown in a lighter colour. This provides a compact visual summary of the fitted network and the associated Wald-type summaries.

The significance threshold can be changed using the `alpha` argument:

```
plotnn(intnn, alpha = 0.01)
```

Intercept terms can also be displayed if desired:

```
plotnn(intnn, intercept = TRUE)
```

Partial covariate-effect plots

The `plot()` method produces PCE plots. For a continuous covariate such as `bmi`, the PCE plot shows how the estimated effect of a d-unit increase in BMI varies across the observed BMI range:

```
plot(intnn, which = "bmi", conf_int = TRUE)
```

If the PCE is approximately constant, the fitted model suggests an approximately linear effect for that covariate. If the PCE varies across the covariate range, the fitted model suggests a non-linear effect. This is one of the main advantages of the PCE plot: it gives an effect-size interpretation while still allowing the fitted neural network to represent non-linear relationships.

For binary covariates, the PCE corresponds to the average change in fitted response when the covariate changes from 0 to 1. For example, the effect of smoking status can be visualised using:

```
plot(intnn, which = "smoker", conf_int = TRUE)
```

Several covariates can be plotted at once:

```
plot(
  intnn,
  which = c("age", "bmi", "children", "smoker"),
  conf_int = TRUE
)
```

The uncertainty intervals are computed using the approximate sampling distribution of the fitted parameters. The default uncertainty method can be changed using the `method` argument. For example, the delta-method intervals are obtained using:

```
plot(
  intnn,
  which = "age",
  conf_int = TRUE,
  method = "delta_method"
)
```

Alternatively, simulation from the approximate sampling distribution of the fitted parameters can be used:

```
plot(
  intnn,
  which = "age",
  conf_int = TRUE,
  method = "mlesim"
)
```

Visualising interactions

Because FNNs can represent interactions between covariates, it can also be useful to examine whether a PCE changes across the values of another covariate. For example, the effect of `bmi` can be plotted separately for different smoking-status groups:

```
plot(
  intnn,
  which = "bmi",
  by = "smoker",
  conf_int = TRUE
)
```

If the resulting PCE curves differ substantially between the two smoking-status groups, this suggests that the fitted network has captured an interaction between `bmi` and `smoker`. This type of plot is exploratory, but it is useful for communicating how the fitted neural network changes across different regions of the covariate space.

Prediction

The `predict()` method returns fitted values for the training data when no new data are supplied:

```
fitted_values <- predict(intnn)

head(fitted_values)
```

Predictions for new observations can be obtained using the `newdata` argument:

```
predict(intnn, newdata = insurance[1:5, ])
```

This gives `"interpretnn"` objects a prediction interface similar to other fitted model objects in R. The same object can therefore be used both for interpretation and prediction.

Summary of the example

This example illustrates the main role of **interpretnn**: it converts a fitted shallow FNN into an object that can be inspected using familiar statistical modelling tools. The fitted network can be printed, summarised, visualised, and used for prediction through standard R methods. The resulting output is not intended to replace predictive validation or the methodological checks discussed in McInerney and Burke (2023). Rather, it provides a practical interface for interpreting fitted shallow FNNs using regression-style summaries and covariate-effect visualisations.

Table 2: Main user-facing functions and methods in 'interpretnn'.

Function	Purpose
'nn_fit()'	Fits a shallow FNN using a formula interface.
'interpretnn()'	Converts a fitted neural-network object into an object of class 'interpretnn'.
'print()'	Prints a compact description of the fitted network and its architecture.
'summary()'	Produces regression-style summaries, including PCE estimates and Wald-type summaries.
'aov()'	Produces covariate-level Wald-type summaries, including grouped summaries for categorical c
'plot()'	Produces partial covariate-effect plots, with optional uncertainty intervals.
'plotnn()'	Produces a neural-network architecture diagram annotated with Wald-type summaries.
'predict()'	Returns fitted values or predictions for new observations.

5 Supported objects and methods

This section summarises the model types, objects, and user-facing methods currently supported by **interpretnn**. The package is organised around the `"interpretnn"` object, which stores the fitted neural-network parameters together with the additional quantities required for statistical summaries, visualisation, and prediction.

Supported model types

The current version of **interpretnn** is designed for single-hidden-layer feedforward neural networks fitted to tabular data. The package supports continuous and binary response variables. For continuous responses, the fitted network is treated as a non-linear Gaussian regression model. For binary responses, the fitted network is treated as a Bernoulli regression model with a logistic output activation.

The main supported architecture is a shallow FNN of the form described in Section 2.2. This scope is deliberate: the inferential summaries implemented in the package are most naturally suited to parsimonious shallow networks, where the model can be interpreted as a flexible extension of classical regression. The package is not intended to provide general-purpose inference for arbitrary deep-learning architectures.

Input variables may be continuous, binary, or categorical. Categorical variables are represented internally through dummy variables in the model matrix. Where possible, **interpretnn** retains the mapping between dummy variables and the original covariates, allowing categorical predictors to be summarised at the covariate level rather than only at the dummy-variable level.

Supported fitting backends

The package includes the convenience function `nn_fit()`, which provides a formula-based interface for fitting shallow FNNs before applying the interpretation tools. This function is intended for standard examples and workflows where the user wants to fit and interpret a simple FNN using familiar R syntax.

The broader design of **interpretnn**, however, is based on post-processing fitted neural networks rather than replacing existing fitting software. The function `interpretnn()` is responsible for converting a fitted neural-network object into an object of class `"interpretnn"`. For fitted objects produced by `nn_fit()`, the required model information is stored automatically. For objects fitted directly using other packages, additional information such as the original formula, model matrix, response vector, response type, or penalty value may be required if it is not stored in the fitted object.

This design separates model fitting from model interpretation. Neural-network optimisation can therefore be handled by existing software, while **interpretnn** focuses on providing statistical summaries and visualisations once a model has been fitted.

User-facing functions and methods

The main user-facing functions and methods are summarised in Table **??**. Together, these functions are designed to make a fitted FNN behave more like a standard statistical model object in R.

The `print()` method gives a concise description of the fitted model, including the model call, response type, sample size, number of input nodes, number of hidden nodes, and penalty value. It is intended as a quick check of the model object and architecture.

The `summary()` method provides the main regression-style output. It reports covariate-level effect estimates, standard errors, Wald-type statistics, and p-values. For categorical predictors, the summary can report effects at the level of the original covariate, rather than requiring the user to interpret each dummy variable separately. Single-parameter summaries for individual weights are also available, but these are best viewed as diagnostic quantities.

The `aov()` method provides analysis-of-variance-style output. For continuous and binary inputs, this is equivalent to the input-level Wald-type summary. For categorical covariates represented by several dummy variables, the method tests the associated input nodes jointly. This is useful when the analyst wants to assess whether a factor contributes to the fitted network as a whole.

The `plot()` method produces PCE plots. For continuous covariates, these plots show how the estimated effect of a change in the covariate varies across the observed covariate range. For binary covariates, the plot summarises the average change in the fitted response when the covariate changes from 0 to 1. Uncertainty intervals can be added using either the delta method or simulation from the approximate sampling distribution of the fitted parameters.

The `plotnn()` function produces a visual representation of the fitted architecture. This plot shows the input nodes, hidden nodes, output node, and fitted connections. It can also highlight statistically significant input nodes and weights using the Wald-type summaries computed by the package.

Finally, the `predict()` method provides a standard prediction interface. When no new data are supplied, it returns fitted values for the training data. When a `newdata` argument is supplied, it returns predictions for the new observations.

Output objects

The main object returned by `interpretnn()` has class `"interpretnn"`. This object contains fitted parameters, data objects, likelihood quantities, covariance estimates, Wald-type summaries, and covariate-effect summaries. Users will typically interact with this object through the methods described above rather than by directly extracting components.

The summary and analysis-of-variance methods return structured output that can be printed, inspected, or stored for later use. Similarly, the plotting methods return graphical objects where appropriate, allowing users to further modify or combine the plots using standard R graphics workflows.

The overall aim is to provide an interface that feels familiar to users of standard statistical modelling functions while allowing the fitted model to remain a neural network. In this way, **interpretnn** acts as a bridge between neural-network fitting software and regression-style statistical interpretation.

6 Relationship to existing software

The R ecosystem contains several packages for fitting neural networks and several packages for interpreting fitted machine-learning models. The contribution of **interpretnn** is different from both of these groups. It is not primarily a neural-network optimiser, and it is not a fully model-agnostic explainability package. Rather, it provides model-based statistical summaries for shallow FNNs by using the fitted neural-network parameterisation directly.

Neural-network fitting packages

Packages such as **nnet** (Venables and Ripley, 2002) and **neuralnet** (Fritsch et al., 2019) provide direct tools for fitting shallow neural networks in R. The **keras** (Allaire and Chollet, 2023) and **torch** (Falbel and Luraschi, 2023) packages provide more general deep-learning frameworks that can be used to fit a much wider range of neural-network architectures. These packages are designed primarily for model fitting, training, and prediction. They typically provide access to fitted values, predictions, training histories, losses, and fitted weights.

By contrast, **interpretnn** focuses on what happens after a shallow FNN has been fitted. Its purpose is to convert a fitted network into an object that can be summarised and visualised using statistical modelling tools. In this sense, **interpretnn** complements neural-network fitting packages rather than competing with them.

The distinction is similar to the distinction between estimating a model and providing a statistical interface for interrogating it. For example, a fitted neural network may contain all of the information needed for prediction, but the user may still want covariate-level summaries, grouped tests for factor variables, uncertainty estimates, or effect plots. These are the outputs that **interpretnn** is designed to provide.

Table 3: Comparison of 'interpretnn' with selected neural-network fitting and model-interpretability packages.

Package	Main role	Fits FNNs	Regression
nnet	Fitting shallow neural networks.	Yes	Limited
neuralnet	Fitting shallow neural networks.	Yes	Limited
keras	Deep-learning interface.	Yes	No
torch	Deep-learning framework.	Yes	No
pdp	Partial dependence and related plots.	No	No
iml	Model-agnostic interpretability.	No	No
DALEX	Model-agnostic explainability.	No	No
vip	Variable importance.	No	No
interpretnn	Statistical post-processing for shallow FNNs.	Via `nn_fit()` convenience wrapper	Yes

Model-agnostic interpretability packages

There are also several R packages for model-agnostic interpretability, including packages that provide partial dependence plots, accumulated local effects, variable-importance measures, surrogate models, or Shapley-value-based explanations. Examples include packages such as **pdp** (**?**), **iml** (**?**), **DALEX** (**?**), and **vip** (**?**).

These tools are valuable because they can be applied to many different fitted model classes. Their model-agnostic nature means that the same interpretability method can often be used for random forests, boosted trees, support vector machines, neural networks, and other predictive models. However, this generality also means that they typically do not use the specific likelihood-based parameterisation of a shallow FNN. They are primarily designed to explain fitted prediction functions rather than to provide regression-style summaries of model parameters.

The approach in **interpretnn** is different. It uses the fitted FNN parameterisation to compute Wald-type summaries, covariance-based standard errors, and PCE uncertainty intervals. The resulting output is therefore closer in spirit to the output from classical statistical models, such as `lm()` and `glm()`, than to purely model-agnostic explainability tools.

This distinction is important. A model-agnostic partial dependence plot can describe how predictions change as a covariate varies, but it does not usually provide a covariate-level Wald-type summary or a regression-style table based on the fitted neural-network parameters. Similarly, a variable-importance measure can rank predictors by predictive contribution, but it does not provide a model-based standard error for a covariate-effect estimate. **interpretnn** is designed to provide these statistically motivated summaries for the specific case of shallow FNNs.

Summary comparison

Table **??** summarises the relationship between **interpretnn** and selected related packages. The table is intended to highlight the role of **interpretnn** as a post-processing and interpretation layer rather than as a replacement for existing neural-network or explainability packages.

The comparison is not intended to suggest that these packages serve the same purpose. Instead, it clarifies the niche filled by **interpretnn**. Neural-network packages provide fitting engines; model-agnostic interpretability packages provide general-purpose explanations of fitted prediction functions; **interpretnn** provides model-based, regression-style summaries for shallow FNNs.

This positioning also motivates the design of the package. Rather than attempting to cover all neural-network architectures or all explainability methods, **interpretnn** focuses on a narrower statistical modelling setting: shallow FNNs fitted to tabular data, where likelihood-based summaries and covariate-effect visualisations can help make the fitted model more accessible to statisticians.

7 Practical guidance and limitations

The summaries produced by **interpretnn** are intended to help users interpret fitted shallow FNNs in a regression-style workflow. However, they should be used with the same care that is required when interpreting any non-linear model, and with additional care due to the particular features of neural-network parameterisations. This section summarises the main practical considerations.

First, **interpretnn** is intended for shallow, parsimonious FNNs fitted to tabular data. The methodology implemented in the package is most appropriate when the fitted network is small enough to be

inspected and when the number of hidden nodes is not excessive relative to the number of covariates and observations. In this setting, the model can be viewed as a flexible extension of classical regression. For large or highly overparameterised networks, the individual parameters are typically much harder to interpret, and covariance-based summaries may become unstable.

Second, the use of a ridge penalty, or weight decay, is recommended. The penalty can improve numerical stability and reduce issues associated with near-singular Hessian matrices. In **interpretnn**, the ridge penalty is also incorporated into the computation of the covariance matrix and related Wald-type summaries. The value of the penalty should therefore be treated as part of the fitted model. The package reports the value of λ used in fitting so that the interpretation of the resulting summaries is tied to the fitted regularised network.

Third, the fitted neural network may depend on the optimisation procedure. Neural-network objective functions are generally non-convex, and different random initialisations can lead to different fitted parameter values. For this reason, users should use multiple random initialisations when fitting the model, inspect whether the fitted loss is stable, and avoid placing too much emphasis on summaries from a single unstable fit. The convenience function `nn_fit()` includes the argument `n_init` for this purpose.

Fourth, individual neural-network weights should be interpreted cautiously. A single input-to-hidden-layer weight is only one component of the fitted function, and its meaning depends on the remaining weights in the network. In addition, hidden nodes can be permuted without changing the fitted input-output function, and sign or label symmetries can lead to equivalent parameterisations. For this reason, **interpretnn** reports individual-weight summaries mainly as diagnostic quantities. In most applications, input-level summaries, covariate-level summaries, and PCE plots will be more useful for interpretation.

Fifth, categorical covariates should generally be interpreted at the covariate level rather than only at the dummy-variable level. When a factor is represented by several dummy variables, separate input-level summaries can be difficult to interpret in isolation. The `aov()` method therefore groups the associated input nodes and provides a covariate-level Wald-type summary. This gives an output closer in spirit to an analysis-of-variance table for a standard regression model.

Sixth, PCE plots should be interpreted as summaries of the fitted model, not as causal effects. A PCE describes how the fitted response changes when a covariate is varied while the remaining covariates are averaged over their observed values. As with partial dependence plots more generally, interpretation can be more difficult when covariates are strongly correlated or when the plot evaluates combinations of covariate values that are rare in the observed data. The plots are therefore best viewed as descriptive tools for understanding the fitted prediction function.

Finally, the inferential summaries produced by **interpretnn** do not replace model checking or predictive validation. Users should still assess predictive performance using appropriate validation procedures, examine the sensitivity of the fitted model to architecture and penalty choices, and consider whether the chosen neural network is appropriate for the scientific or applied question. The package is designed to make fitted shallow FNNs more interpretable, but the quality of the resulting interpretation depends on the quality and stability of the fitted model.

8 Conclusion

The **interpretnn** package provides a statistical post-processing layer for shallow feedforward neural networks in R. Rather than introducing a new neural-network optimiser, the package converts fitted FNNs into a common `"interpretnn"` object and provides familiar methods for inspecting, summarising, visualising, and predicting from the fitted model. In this way, **interpretnn** aims to make shallow neural networks behave more like standard statistical model objects in R.

The package implements regression-style summaries, Wald-type summaries for individual weights and groups of weights, analysis-of-variance-style summaries for covariates, PCE plots with uncertainty estimates, architecture diagrams annotated with statistical summaries, and prediction methods for new observations. These tools are designed to complement existing neural-network fitting packages by providing outputs that are more familiar to users of classical regression models.

The statistical methodology underlying these summaries is developed in **?**. The contribution of the present article is to describe the corresponding R implementation and demonstrate how the methodology can be used in practice through a coherent package workflow. By combining neural-network flexibility with regression-style output, **interpretnn** provides a practical interface for statisticians who wish to use shallow FNNs as interpretable statistical modelling tools rather than purely predictive algorithms.

Future development of the package may include broader support for additional fitting backends,

further response distributions, alternative uncertainty estimation methods, and additional diagnostic tools for assessing model stability. More generally, **interpretnn** provides a foundation for integrating shallow neural networks more naturally into the statistical modelling workflow in R.

9 Acknowledgements

This publication has emanated from research conducted with the financial support of Science Foundation Ireland under Grant number 18/CRT/6049. The second author was supported by the Confirm Smart Manufacturing Centre (https://confirm.ie/) funded by Science Foundation Ireland (Grant Number: 16/RC/3918). For the purpose of Open Access, the authors have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

Bibliography

J. Allaire and F. Chollet. *keras: R Interface to 'Keras'*, 2023. R package version 2.11.1. [p1, 12]

D. Falbel and J. Luraschi. *torch: Tensors and Neural Networks with 'GPU' Acceleration*, 2023. R package version 0.10.0. [p1, 12]

S. Fritsch, F. Guenther, and M. N. Wright. *neuralnet: Training of Neural Networks*, 2019. R package version 1.44.2. [p1, 12]

A. McInerney and K. Burke. Feedforward neural networks as statistical models: Improving interpretability through uncertainty quantification. 2023. [p1, 2, 3, 10]

W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. Springer, New York, fourth edition, 2002. ISBN 0-387-95457-0. [p1, 12]

Andrew McInerney
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
ORCiD: 0000-0002-2348-2159
andrew.mcinerney@ul.ie

Kevin Burke
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
kevin.burke@ul.ie